

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра ІІІ

Звіт

з лабораторної роботи №3 з дисципліни
«Системне програмне забезпечення»

„Програмування розгалужених алгоритмів”

Виконав ІІ-45 Янов Б. Є.

Перевірив Бабич Б. Б.

Київ 2026

Лабораторна робота №3

Програмування розгалужених алгоритмів

Завдання:

1. Можливість введення користувачем значень x , y , t , a , b за необхідності.
2. Обчислювати значення функції за введеними значеннями.
3. Виводити на екран результат обчислень.
4. Якщо є ділення, то результат дозволяється виводити: окремо цілу частину та остачу.
5. Програма повинна мати захист від некоректного введення вхідних даних (символи, переповнення, ділення на 0 і т.і.)

Номер завдання – 19

$$Z = \begin{cases} ax^2 + b / x & \text{якщо } x > 0 \\ a + 2b & \text{якщо } x = 0 \\ ax^2 - bx & \text{якщо } x < 0 \end{cases}$$

Текст програми

```
.model small
JUMPS
.stack 100h

.data
; Input messages
msg_a      db 'Enter a: $'
msg_b      db 13, 10, 'Enter b: $'
msg_x      db 13, 10, 'Enter x: $'

; Output messages
msg_res     db 13, 10, 'Result Z = $'
msg_rem     db ' (rem: $'
msg_rem_end db ')$'

; Error messages
msg_err     db 13, 10, 'Error: Invalid input! (Numbers only)$'
msg_ovf     db 13, 10, 'Error: Overflow/Underflow! Limit to 16-
bit.$'

; Variables
buffer      db 7, 0, 7 dup(0)
val_a       dw 0
val_b       dw 0
val_x       dw 0
is_neg      db 0
res_rem     dw 0

.code
main proc
    mov ax, @data
```

```

        mov     ds, ax

        ; Input a
        lea     dx, msg_a
        mov     ah, 09h
        int     21h
        call    InputProc
        jc      exit_prog          ; Exit on error
        mov     val_a, ax

        ; Input b
        lea     dx, msg_b
        mov     ah, 09h
        int     21h
        call    InputProc
        jc      exit_prog          ; Exit on error
        mov     val_b, ax

        ; Input x
        lea     dx, msg_x
        mov     ah, 09h
        int     21h
        call    InputProc
        jc      exit_prog          ; Exit on error
        mov     val_x, ax

        mov     ax, val_x
        cmp     ax, 0
        jg      calc_greater       ; x > 0
        je      calc_equal         ; x == 0
        jl      calc_less          ; x < 0

; x > 0 => ax^2 + b/x
calc_greater:
        ; Calculate ax^2
        mov     ax, val_x
        imul    ax                 ; ax = x^2 (dx:ax)
        jo      overflow_err
        imul    val_a              ; ax = a * x^2
        jo      overflow_err
        mov     bx, ax             ; bx = ax^2

        ; Calculate b/x
        mov     ax, val_b
        cwd                                ; Extend ax to dx:ax for
idiv                                ; ax, dx = rem
        idiv    val_x

        ; Calculate (ax^2) + (b/x)
        add     ax, bx

```

```

        jo    overflow_err

        ; Save remaining
        mov   res_rem, dx
        jmp   print_res

; x = 0 => a + 2b
calc_equal:
        mov   ax, val_b
        mov   cx, 2
        imul  cx                ; ax = 2b
        jo    overflow_err

        add   ax, val_a        ; ax = a + 2b
        jo    overflow_err

        jmp   print_res

; x < 0 => ax^2 - bx
calc_less:
        ; Calculate ax^2
        mov   ax, val_x
        imul  ax
        jo    overflow_err
        imul  val_a
        jo    overflow_err
        mov   bx, ax            ; bx = ax^2

        ; Calculate bx
        mov   ax, val_b
        imul  val_x
        jo    overflow_err      ; ax = bx

        ; Calculate ax^2 - bx
        sub   bx, ax
        jo    overflow_err

        mov   ax, bx
        jmp   print_res

print_res:
        push  ax                ; Save result Z
        lea   dx, msg_res
        mov   ah, 09h
        int   21h
        pop   ax
        call  OutputProc        ; Print first part

        cmp   res_rem, 0        ; Skip if 0

```

```

        je     exit_prog

        ; Print rem
        lea    dx, msg_rem
        mov    ah, 09h
        int    21h

        mov    ax, res_rem
        call   OutputProc

        lea    dx, msg_rem_end
        mov    ah, 09h
        int    21h
        jmp    exit_prog

overflow_err:
        lea    dx, msg_ovf
        mov    ah, 09h
        int    21h
        jmp    exit_prog

exit_prog:
        mov    ax, 4c00h
        int    21h

main endp

```

```

; Copied from lab2
; Returns in AX

```

```

InputProc proc
        lea    dx, buffer
        mov    ah, 0Ah
        int    21h

        lea    si, buffer+2
        mov    cl, buffer+1
        xor    ch, ch
        cmp    cx, 0
        je     input_error

        xor    bx, bx
        mov    is_neg, 0

        mov    al, [si]
        cmp    al, '-'
        jne    check_plus
        mov    is_neg, 1

```

```

        inc    si
        dec    cx
        jmp    parse_loop

check_plus:
        cmp    al, '+'
        jne    parse_loop
        inc    si
        dec    cx

parse_loop:
        cmp    cx, 0
        je     parse_done

        mov    al, [si]
        cmp    al, '0'
        jl     input_error
        cmp    al, '9'
        jg     input_error

        sub    al, '0'
        xor    ah, ah
        push   cx

        mov    cx, ax
        mov    ax, bx
        mov    dx, 10
        imul   dx
        jo     ovf_error

        add    ax, cx
        jo     ovf_error

        mov    bx, ax
        pop    cx

        inc    si
        dec    cx
        jmp    parse_loop

parse_done:
        cmp    is_neg, 1
        jne    save_number
        neg    bx
        jo     ovf_error

save_number:
        mov    ax, bx                ; Return in AX
        cld
        ret

```

```

ovf_error:
    pop    cx
    lea    dx, msg_ovf
    mov    ah, 09h
    int    21h
    stc
    ret

input_error:
    lea    dx, msg_err
    mov    ah, 09h
    int    21h
    stc
    ret

InputProc endp

OutputProc proc
    cmp    ax, 0
    jge    start_convert

    push   ax
    mov    ah, 02h
    mov    dl, '-'
    int    21h
    pop    ax
    neg    ax

start_convert:
    xor    cx, cx
    mov    bx, 10

divide_loop:
    xor    dx, dx
    div    bx
    push   dx
    inc    cx
    test   ax, ax
    jnz    divide_loop

print_loop:
    pop    dx
    add    dl, '0'
    mov    ah, 02h
    int    21h
    loop   print_loop

    ret

OutputProc endp

end main

```

Вміст .lst файлу

Turbo Assembler

Version 4.0

05/23/26 24:17:28

Page

1

lab.asm

```

1      0000                      .model small
2                                  JUMPS
3      0000                      .stack 100h
4
5      0000                      .data
6                                  ; Input messages
7      0000  45 6E 74 65 72 20 61+  msg_a      db  'Enter
a: $'
8          3A 20 24
9      000A  0D 0A 45 6E 74 65 72+  msg_b      db  13, 10,
'Enter b: $'
10         20 62 3A 20 24
11      0016  0D 0A 45 6E 74 65 72+  msg_x      db  13, 10,
'Enter x: $'
12         20 78 3A 20 24
13
14                                  ; Output messages
15      0022  0D 0A 52 65 73 75 6C+  msg_res     db  13, 10,
'Result  Z = $'
16         74 20 5A 20 3D 20 24
17      0030  20 28 72 65 6D 3A 20+  msg_rem     db  ' (rem:
$'
18         24
19      0038  29 24                      msg_rem_end db  ')$'
20
21                                  ; Error messages
22      003A  0D 0A 45 72 72 6F 72+  msg_err     db  13, 10,
'Error:  Invalid  input! (Numbers  only)$'
23         3A 20 49 6E 76 61 6C+
24         69 64 20 69 6E 70 75+
25         74 21 20 28 4E 75 6D+
26         62 65 72 73 20 6F 6E+
27         6C 79 29 24
28      0061  0D 0A 45 72 72 6F 72+  msg_ovf     db  13, 10,
'Error:  Overflow/Underflow! Limit to 16-bit.$'
29         3A 20 4F 76 65 72 66+
30         6C 6F 77 2F 55 6E 64+
31         65 72 66 6C 6F 77 21+
32         20 4C 69 6D 69 74 20+
33         74 6F 20 31 36 2D 62+
34         69 74 2E 24
35
36                                  ; Variables

```

```

37    008F  07 00 07*(00)          buffer          db    7, 0, 7
dup(0)
38    0098  0000                      val_a          dw    0
39    009A  0000                      val_b          dw    0
40    009C  0000                      val_x          dw    0
41    009E  00                      is_neg         db    0
42    009F  0000                      res_rem        dw    0
43
44    00A1                      .code
45    0000                      main proc
46    0000  B8 0000s                      mov  ax,      @data
47    0003  8E D8                      mov  ds,      ax
48
49                                ; Input a
50    0005  BA 0000r                      lea  dx,      msg_a
51    0008  B4 09                      mov  ah,      09h
52    000A  CD 21                      int  21h
53    000C  E8 00EE                      call InputProc
54    000F  73 03 E9 00E4                jc   exit_prog
        ; Exit on error
55    0014  A3 0098r                      mov  val_a, ax
56
57                                ; Input b

```

2

lab.asm

```

58      0017  BA 000Ar          lea  dx,      msg_b
59      001A  B4 09          mov  ah,      09h
60      001C  CD 21          int  21h
61      001E  E8 00DC        call InputProc
62      0021  73 03 E9 00D2    jc   exit_prog
        ; Exit on error
63      0026  A3 009Ar          mov  val_b, ax
64
65                                ; Input x
66      0029  BA 0016r          lea  dx,      msg_x
67      002C  B4 09          mov  ah,      09h
68      002E  CD 21          int  21h
69      0030  E8 00CA        call InputProc
70      0033  73 03 E9 00C0    jc   exit_prog
        ; Exit on error
71      0038  A3 009Cr          mov  val_x, ax
72
73      003B  A1 009Cr          mov  ax,      val_x
74      003E  3D 0000        cmp  ax,      0
75      0041  7F 0D 90 90 90    jg
calc_greater      ; x > 0
76      0046  74 33 90 90 90    je   calc_equal
        ; x == 0
77      004B  7C 47 90 90 90    jl   calc_less
        ; x < 0
78
79                                ; x > 0 => ax^2 + b/x
80      0050          calc_greater:
81                                ; Calculate ax^2
82      0050  A1 009Cr          mov  ax,      val_x
83      0053  F7 E8          imul ax          ;
ax = x^2 (dx:ax)
84      0055  71 03 E9 0094    jo   overflow_err
85      005A  F7 2E 0098r      imul val_a          ;
ax = a
  * x^2
86      005E  71 03 E9 008B    jo   overflow_err
87      0063  8B D8          mov  bx,      ax
        ; bx = ax^2
88
89                                ; Calculate b/x
90      0065  A1 009Ar          mov  ax,      val_b
91      0068  99          cwd          ;
Extend  ax to dx:ax for      idiv
92      0069  F7 3E 009Cr      idiv val_x          ;

```


3

lab.asm

```
115    0094                      calc_less:
116                                ; Calculate ax^2
117    0094  A1 009Cr             mov ax,      val_x
118    0097  F7 E8               imul ax
119    0099  70 53 90 90 90      jo
overflow_err
120    009E  F7 2E 0098r         imul val_a
121    00A2  70 4A 90 90 90      jo
overflow_err
122    00A7  8B D8               mov bx,      ax
                                ; bx = ax^2
123
124                                ; Calculate bx
125    00A9  A1 009Ar             mov ax,      val_b
126    00AC  F7 2E 009Cr         imul val_x
127    00B0  70 3C 90 90 90      jo
overflow_err                      ; ax = bx
128
129                                ; Calculate ax^2 - bx
130    00B5  2B D8               sub bx,      ax
131    00B7  70 35 90 90 90      jo
overflow_err
132
133    00BC  8B C3               mov ax,      bx
134    00BE  EB 01 90           jmp print_res
135
136
137    00C1                      print_res:
138    00C1  50                 push ax                      ; Save
result Z
139    00C2  BA 0022r           lea dx,      msg_res
140    00C5  B4 09             mov ah,      09h
141    00C7  CD 21             int 21h
142    00C9  58                 pop ax
143    00CA  E8 00CC           call OutputProc
                                ; Print first part
144
145    00CD  83 3E 009Fr 00      cmp res_rem, 0
                                ; Skip if 0
146    00D2  74 24 90 90 90      je exit_prog
147
148                                ; Print rem
149    00D7  BA 0030r           lea dx,      msg_rem
150    00DA  B4 09             mov ah,      09h
```

151	00DC	CD 21		int 21h
152				
153	00DE	A1 009Fr		mov ax, res_rem
154	00E1	E8 00B5		call OutputProc
155				
156	00E4	BA 0038r		lea dx,
		msg_rem_end		
157	00E7	B4 09		mov ah, 09h
158	00E9	CD 21		int 21h
159	00EB	EB 0B 90		jmp exit_prog
160				
161				
162	00EE		overflow_err:	
163	00EE	BA 0061r		lea dx, msg_ovf
164	00F1	B4 09		mov ah, 09h
165	00F3	CD 21		int 21h
166	00F5	EB 01 90		jmp exit_prog
167				
168	00F8		exit_prog:	
169	00F8	B8 4C00		mov ax, 4c00h
170	00FB	CD 21		int 21h
171	00FD		main endp	

4

lab.asm

```
172
173
174
175             ; Copied from lab2
176             ; Returns in AX
177
178 00FD             InputProc proc
179 00FD BA 008Fr             lea dx, buffer
180 0100 B4 0A             mov ah, 0Ah
181 0102 CD 21             int 21h
182
183 0104 BE 0091r             lea si, buffer+2
184 0107 8A 0E 0090r             mov cl, buffer+1
185 010B 32 ED             xor ch, ch
186 010D 83 F9 00             cmp cx, 0
187 0110 74 7E 90 90 90             je input_error
188
189 0115 33 DB             xor bx, bx
190 0117 C6 06 009Er 00             mov is_neg, 0
191
192 011C 8A 04             mov al, [si]
193 011E 3C 2D             cmp al, '-'
194 0120 75 0D 90 90 90             jne check_plus
195 0125 C6 06 009Er 01             mov is_neg, 1
196 012A 46             inc si
197 012B 49             dec cx
198 012C EB 0A 90             jmp parse_loop
199
200 012F             check_plus:
201 012F 3C 2B             cmp al, '+'
202 0131 75 05 90 90 90             jne parse_loop
203 0136 46             inc si
204 0137 49             dec cx
205
206 0138             parse_loop:
207 0138 83 F9 00             cmp cx, 0
208 013B 74 34 90 90 90             je parse_done
209
210 0140 8A 04             mov al, [si]
211 0142 3C 30             cmp al, '0'
212 0144 7C 4A 90 90 90             jl input_error
213 0149 3C 39             cmp al, '9'
214 014B 7F 43 90 90 90             jg input_error
215
```

216	0150	2C 30	sub	al,	'0'
217	0152	32 E4	xor	ah,	ah
218	0154	51	push	cx	
219					
220	0155	8B C8	mov	cx,	ax
221	0157	8B C3	mov	ax,	bx
222	0159	BA 000A	mov	dx,	10
223	015C	F7 EA	imul	dx	
224	015E	70 26 90 90 90		jo	ovf_error
225					
226	0163	03 C1	add	ax,	cx
227	0165	70 1F 90 90 90		jo	ovf_error
228					

5

lab.asm

```
229 016A 8B D8          mov  bx,      ax
230 016C 59            pop  cx
231
232 016D 46            inc  si
233 016E 49            dec  cx
234 016F EB C7          jmp  parse_loop
235
236 0171              parse_done:
237 0171 80 3E 009Er 01      cmp  is_neg, 1
238 0176 75 0A 90 90 90      jne  save_number
239 017B F7 DB          neg  bx
240 017D 70 07 90 90 90      jo   ovf_error
241
242 0182              save_number:
243 0182 8B C3          mov  ax,      bx
      ; Return      in AX
244 0184 F8            clc
245 0185 C3            ret
246
247 0186              ovf_error:
248 0186 59            pop  cx
249 0187 BA 0061r        lea  dx,      msg_ovf
250 018A B4 09          mov  ah,      09h
251 018C CD 21          int  21h
252 018E F9            stc
253 018F C3            ret
254
255 0190              input_error:
256 0190 BA 003Ar        lea  dx,      msg_err
257 0193 B4 09          mov  ah,      09h
258 0195 CD 21          int  21h
259 0197 F9            stc
260 0198 C3            ret
261 0199              InputProc endp
262
263 0199              OutputProc  proc
264 0199 3D 0000          cmp  ax,      0
265 019C 7D 0D 90 90 90      jge
start_convert
266
267 01A1 50            push ax
268 01A2 B4 02          mov  ah,      02h
269 01A4 B2 2D          mov  dl,      '-'
270 01A6 CD 21          int  21h
```

271	01A8	58	pop	ax	
272	01A9	F7 D8	neg	ax	
273					
274	01AB		start_convert:		
275	01AB	33 C9	xor	cx,	cx
276	01AD	BB 000A	mov	bx,	10
277	01B0		divide_loop:		
278	01B0	33 D2	xor	dx,	dx
279	01B2	F7 F3	div	bx	
280	01B4	52	push	dx	
281	01B5	41	inc	cx	
282	01B6	85 C0	test	ax,	ax
283	01B8	75 F6	jnz	divide_loop	
284					
285	01BA		print_loop:		

6

lab.asm

```
286    01BA  5A                                pop    dx
287    01BB  80 C2 30                        add    dl,    '0'
288    01BE  B4 02                        mov    ah,    02h
289    01C0  CD 21                        int     21h
290    01C2  E2 F6                        loop   print_loop
291
292    01C4  C3                                ret
293    01C5                                OutputProc    endp
294
295                                end main
```

Symbol Name	Type	Value
??DATE	Text	"05/23/26"
??FILENAME	Text	"lab"
??TIME	Text	"24:17:27"
??VERSION	Number	0400
@32BIT	Text	0
@CODE	Text	_TEXT
@CODESIZE	Text	0
@CPU	Text	0101H
@CURSEG	Text	_TEXT
@DATA	Text	DGROUP
@DATASIZE	Text	0
@FILENAME	Text	LAB
@INTERFACE	Text	00H
@MODEL	Text	2
@STACK	Text	DGROUP
@WORDSIZE	Text	2
BUFFER	Byte	DGROUP:008F
CALC_EQUAL	Near	_TEXT:007B
CALC_GREATER	Near	_TEXT:0050
CALC_LESS	Near	_TEXT:0094
CHECK_PLUS	Near	_TEXT:012F
DIVIDE_LOOP	Near	_TEXT:01B0
EXIT_PROG	Near	_TEXT:00F8
INPUTPROC	Near	_TEXT:00FD
INPUT_ERROR	Near	_TEXT:0190
IS_NEG	Byte	DGROUP:009E
MAIN	Near	_TEXT:0000
MSG_A	Byte	DGROUP:0000
MSG_B	Byte	DGROUP:000A
MSG_ERR	Byte	DGROUP:003A
MSG_OVF	Byte	DGROUP:0061
MSG_REM	Byte	DGROUP:0030
MSG_REM_END	Byte	DGROUP:0038
MSG_RES	Byte	DGROUP:0022
MSG_X	Byte	DGROUP:0016
OUTPUTPROC	Near	_TEXT:0199
OVERFLOW_ERR	Near	_TEXT:00EE
OVF_ERROR	Near	_TEXT:0186
PARSE_DONE	Near	_TEXT:0171
PARSE_LOOP	Near	_TEXT:0138
PRINT_LOOP	Near	_TEXT:01BA

PRINT_RES	Near	_TEXT:00C1
RES_REM	Word	DGROUP:009F
SAVE_NUMBER	Near	_TEXT:0182
START_CONVERT	Near	_TEXT:01AB
VAL_A	Word	DGROUP:0098
VAL_B	Word	DGROUP:009A
VAL_X	Word	DGROUP:009C

Groups & Segments	Bit	Size	Align	Combine	Class
-------------------	-----	------	-------	---------	-------

DGROUP	Group				
STACK	16	0100	Para	Stack	STACK
_DATA	16	00A1	Word	Public	DATA

_TEXT

16 01C5 Word

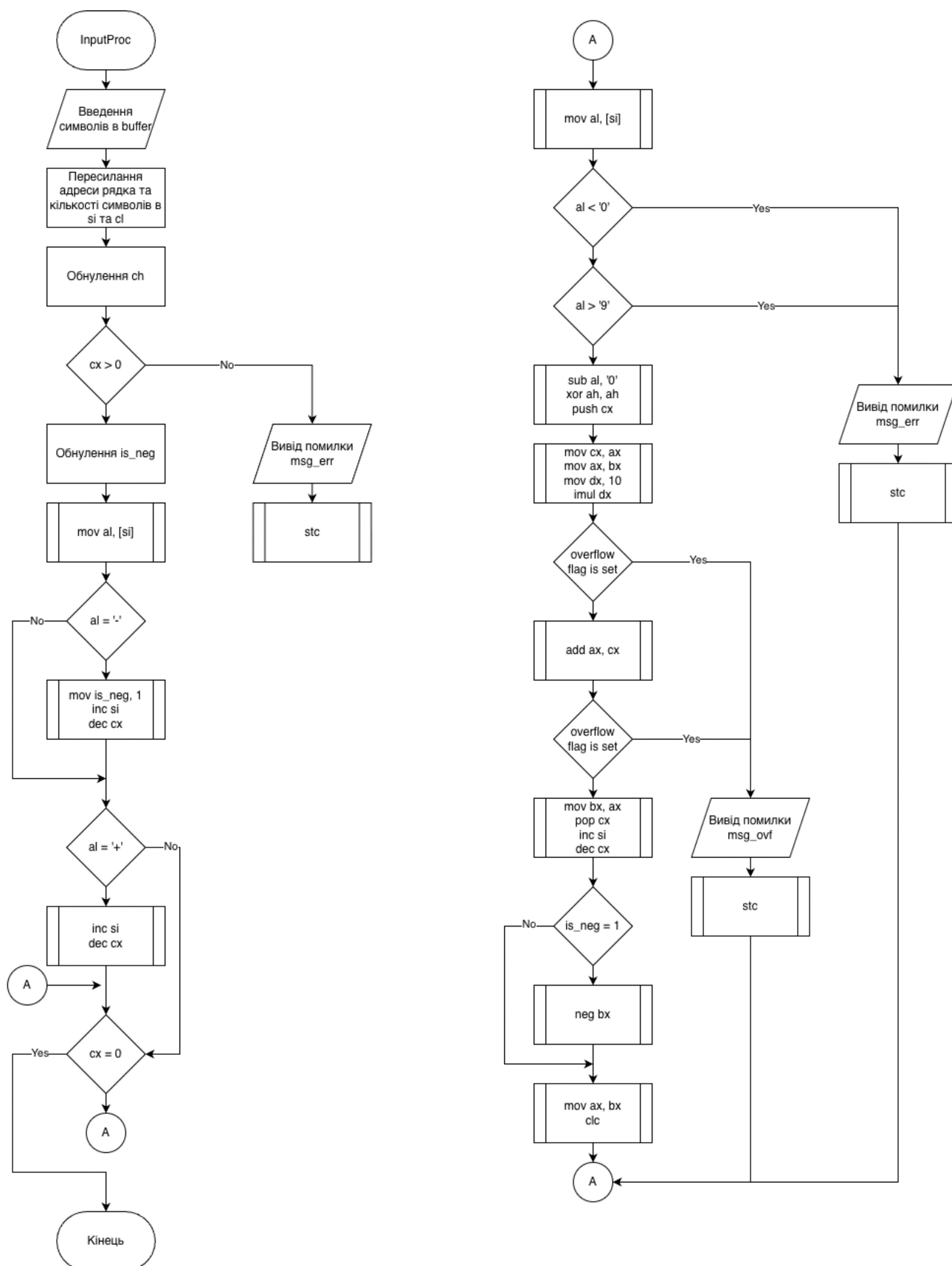
Public CODE

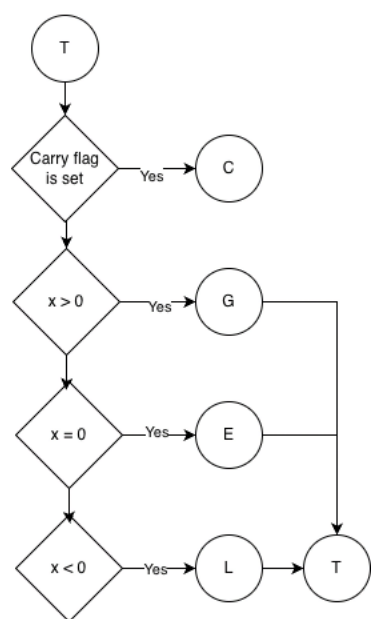
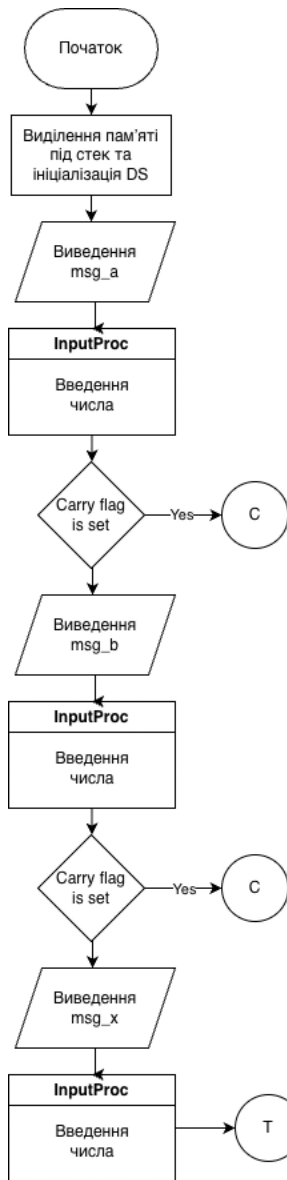
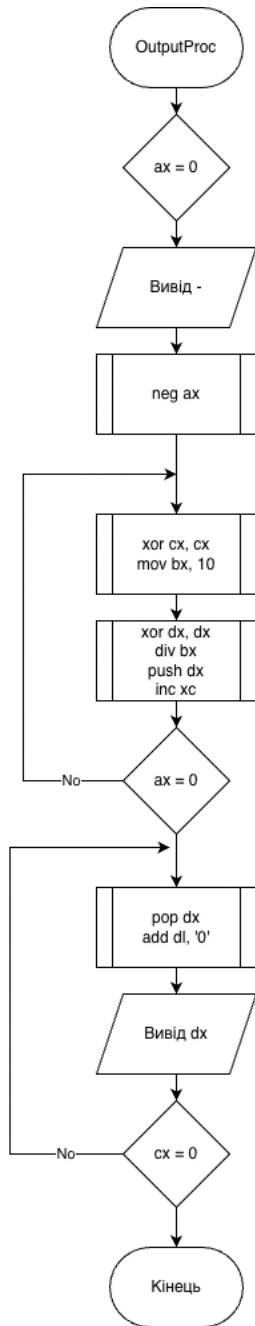
Вміст .map файлу

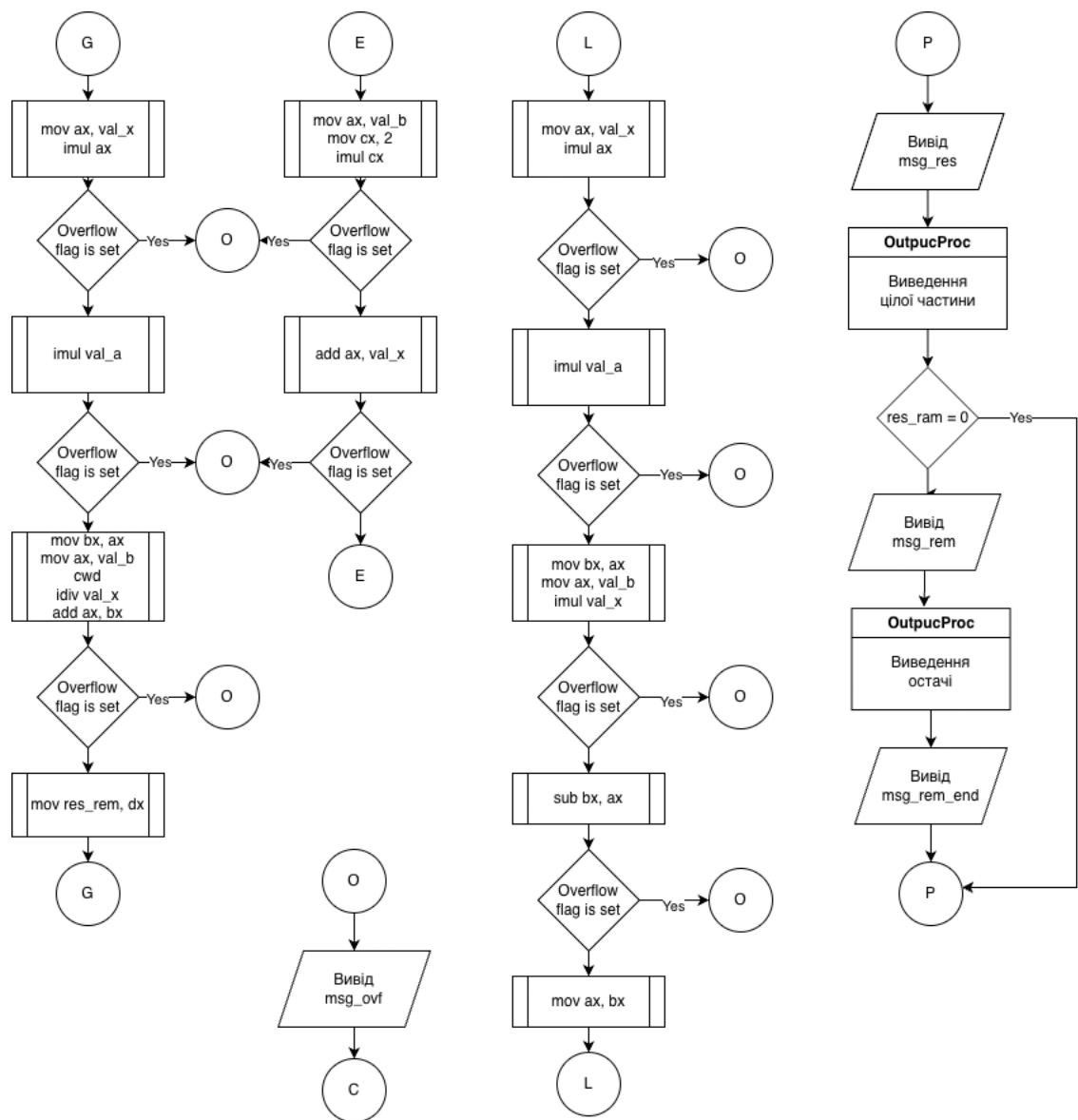
Start	Stop	Length	Name	Class
00000H	001C4H	001C5H	_TEXT	CODE
001D0H	00270H	000A1H	_DATA	DATA
00280H	0037FH	00100H	STACK	STACK

Program entry point at 0000:0000

Блок схема програми







Приклад роботи програми

```
C:\LAB3>LAB.EXE
Enter a: 5
Enter b: 10
Enter x: 15
Result Z = 1125 (rem: 10)
C:\LAB3>LAB.EXE
Enter a: 5
Enter b: 10
Enter x: -15
Result Z = 1275
C:\LAB3>LAB.EXE
Enter a: 5
Enter b: 10
Enter x: 0
Result Z = 25
C:\LAB3>_
```

```
C:\LAB3>LAB.EXE
Enter a: fif
Error: Invalid input! (Numbers only)
C:\LAB3>_
```

Висновок

1. Реалізовано програму з введенням значень a , b та x .
2. Реалізовано виконання математичних дій над числами в залежності від варіанту.
3. Реалізовано вивід чисел з остачею.
4. Програма має захист від некоректного введення вхідних даних.